

Trusted Credentials, Untrusted Behavior: Benchmarking LLM-Agent Security in High-Performance Computing *

Jie Li
Texas Tech University
jie.li@ttu.edu

July 29, 2026

Abstract

Large language model (LLM) agents are starting to take on routine work in high-performance computing (HPC), including monitoring Slurm jobs, diagnosing failed builds, inspecting simulation output, and coordinating scientific workflows. To do this work, an agent commonly acts under its user’s credentials and inherits the user’s access to files and the scheduler. This arrangement creates a failure mode that ordinary account-level controls do not capture. Adversarial instructions in a log, tool description, shared file, or peer-agent message may redirect the agent beyond the task the user assigned, even though every resulting command is authenticated and permitted for that account. We refer to this as the *hijacked authorized agent* problem. Existing agent-security studies explain relevant mechanisms, such as indirect prompt injection and tool misuse, but generally evaluate them in web, enterprise, or personal-assistant settings. HPC security, by contrast, has mature controls for identity and isolation but does not ordinarily represent the intent of a particular task. This paper defines the threat model in the HPC setting, identifies attack surfaces created by schedulers, shared storage, multi-project accounts, and scientific workflows, and examines where current controls fall short. It concludes with a research agenda and a plan for an empirical benchmark, TaskBound.

1 Introduction

Much of the effort involved in using an HPC system happens around the batch job itself. Users inspect logs, repair scripts, install dependencies, sort through output, and decide what to run next. These are natural tasks for an LLM agent with access to files, command-line tools, and scheduler commands. Several recent systems connect agents to HPC resources and scientific workflows [1–3]; another uses multiple agents to build and repair HPC software [4]. Adoption is also occurring outside research prototypes. Aalto Scientific Computing, for example, has published guidance for users running agentic coding tools on its Triton cluster [5].

Giving an agent enough access to be useful also gives untrusted material a new route to that access. A debugging agent must read the output of a failed job. A build agent must interpret compiler messages and package metadata. A workflow agent may exchange files or messages with other agents. If any of this material contains adversarial instructions, the agent may act on them

*Position/vision paper.

using the authority it received from the user. The resulting commands can be valid from the operating system’s point of view and still be outside the user’s intended task.

Agent-security research provides a good account of the underlying mechanisms. Indirect prompt injection can manipulate applications that consume untrusted content [6]; tool-using agents can take harmful actions [7]; and benchmarks can measure both task completion and resistance to attack [8]. Tool descriptions and inter-agent protocols add further avenues for manipulation [9–11]. Most evaluations, however, use settings such as email, banking, travel, or generic tool sandboxes. They do not model scheduler allocations, parallel filesystems, or accounts that legitimately span several scientific projects.

HPC security starts from a different concern: deciding which principals may use which resources. Zone-based architectures separate access, management, compute, and storage functions [12]. Deployed systems isolate users’ processes, files, network traffic, and accelerators [13], while federated identity and zero-trust designs strengthen authentication and access control across facilities [14]. These controls remain necessary. Yet an access check cannot determine whether a permitted command reflects the user’s request or an instruction copied from a poisoned log into the agent’s context.

Our concern lies between these two lines of work. The identity and credentials may be valid, and each action may pass the site’s authorization checks, while the sequence of actions no longer serves the task the user assigned. We call an agent in this state a *hijacked authorized agent*.

1.1 Position

Our position rests on three observations:

1. HPC changes the practical meaning of a successful prompt-injection attack. Scheduler authority, durable shared storage, multi-project access, and scientific provenance introduce consequences that are absent from many current agent benchmarks.
2. Neither neighboring literature measures the complete problem. Agent-security benchmarks omit important HPC resources and policies, while HPC controls do not test whether adversarial content can redirect an authenticated agent.
3. The question is already relevant to deployed systems. Agents are being used in HPC workflows [1, 2, 4], and at least one computing center has begun documenting the associated risks [5]. Evaluation should develop alongside adoption rather than after local practices have hardened.

1.2 Contributions

1. We define the **hijacked authorized agent** threat model and distinguish it from privilege escalation and generic indirect prompt injection (Section 4).
2. We organize the relevant HPC attack surfaces into five groups: shared-filesystem poisoning, scheduler and log channels, tool/module/MCP poisoning, cross-project leakage, and multi-agent exfiltration (Section 5).
3. We compare these surfaces with existing HPC and agent-monitoring controls (Section 6).
4. We identify open research problems and describe TaskBound, a companion benchmark now under development (Section 7).

1.3 Scope

This is a threat characterization and research agenda, not an empirical evaluation. We do not report attack success rates, assess a particular model or framework, or present a complete defense. Training-time poisoning, weight extraction, GPU side channels, kernel or hypervisor compromise, and general content-safety jailbreaks are also outside our scope. The narrower question here is how content from the HPC environment can redirect an agent that already holds valid credentials.

2 Background

2.1 LLM agents are entering HPC

Several projects now place LLM agents in direct contact with HPC infrastructure. Ma et al. connect LangChain and LangGraph agents to HPC resources through Parsl and demonstrate molecular-dynamics workflows on ALCF’s Polaris system [1]. Rosendo et al. combine an LLM interface, multi-agent decision making, facility APIs, and provenance services for experiments spanning two ORNL user facilities [2]. Mondesire et al. use a multi-agent system to build and repair more than 200 HPC software packages [4]. In the last example, compiler errors and build logs are not incidental output; they are inputs to the next decision made by the agent.

Dawson et al. identify correctness, reproducibility, and safe resource use as open problems for agentic supercomputing workflows [3]. Similar concerns have reached site documentation. Aalto Scientific Computing notes the use of Claude Code and OpenAI Codex for coding and Slurm job management on Triton, and warns users about prompt injection, malicious skills, external code, and confidentiality [5]. The systems literature nevertheless remains focused mainly on capability, orchestration, and provenance. It offers little evidence about behavior when routine inputs such as logs or tool output are malicious.

2.2 LLM-agent security: known attack mechanisms

The underlying attack mechanisms have been studied mostly outside HPC. Greshake et al. showed that instructions embedded in retrieved content, rather than supplied by the user, can change an LLM-integrated application’s behavior and lead to data disclosure or persistent compromise [6]. ToolEmu evaluates harmful actions by tool-using agents in an LM-emulated sandbox [7]. AgentDojo provides a reproducible benchmark with 97 tasks and 629 security test cases across email, banking, travel, and workspace applications [8]. Its joint measurement of task utility and attack success is particularly useful for our proposed benchmark (Section 7.2).

Recent work broadens the picture. Ferrag et al. survey attacks that include inter-agent protocol exploits [10], and Wang et al. systematize prompt-injection threats and their defenses [11]. Shi et al. show that a malicious tool description can steer an agent toward an attacker-controlled tool without requiring white-box access to the model [9]. This result is relevant to environment modules, shared utilities, and MCP tool servers (Section 5.3). Still, an email inbox or travel itinerary does not exercise scheduler authority, project allocations, or filesystem state shared across jobs and users. Those differences affect both the available attacks and the definition of a policy violation.

2.3 HPC security: established controls and their assumptions

HPC security has developed strong controls around identities and resources. NIST SP 800-223 organizes HPC infrastructure into access, management, compute, and storage zones and analyzes threats to each [12]. Prout et al. describe user separation at the MIT Lincoln Laboratory Supercomputing

Center across processes, files, network traffic, and accelerators [13]. Alam et al. present a federated single-sign-on and zero-trust design for AI and HPC research infrastructure, including multi-factor authentication and time-limited role-based access [14].

These mechanisms determine who may access a resource and under what conditions. They generally treat a successful authorization decision as sufficient evidence that the action is allowed. For an LLM agent, that inference is incomplete. A POSIX check or scheduler record cannot tell whether the user requested an action or whether the agent derived it from adversarial text in a log. The site sees the same account in either case. The missing information is the scope and origin of the task that led to the action.

3 What Existing Work Leaves Open

The three bodies of work reviewed above stop at different boundaries. Agentic-HPC systems demonstrate useful access to schedulers, files, and scientific instruments, but seldom evaluate hostile inputs encountered during a workflow. Agent-security benchmarks do evaluate hostile inputs, though without HPC resources or policies. HPC security protects those resources, but its enforcement point is usually the user or account rather than the narrower task delegated to an agent.

What remains poorly characterized is therefore a common and plausible case: an agent holds valid HPC credentials, reads content in the course of authorized work, and is redirected into a different but still account-permitted action. We have not found prior work that treats this intersection as its primary object of study. The rest of the paper gives the problem a threat model, identifies its HPC-specific attack surfaces, and asks what an evaluation would need to measure.

4 Threat Model: The Hijacked Authorized Agent

4.1 Definition

We call an agent **hijacked** when content encountered during ordinary operation causes it to act outside its assigned task, even though the user’s account and the agent’s tool configuration permit each individual action. The content might appear in a file, a job log, tool output, or a message from another agent. No conventional privilege escalation is required: the attacker need not obtain root access, bypass POSIX permissions, or steal credentials.

Consider an agent asked to diagnose a failed job in project A. The user may also belong to project B and may be allowed to submit jobs to several partitions. If a poisoned log leads the agent to read project B or launch an unrelated job, the operating system and scheduler may record nothing more than permitted activity by that user. The violated boundary is the authority implied by the debugging task, not the standing authority of the account. Conventional HPC controls do not usually encode that task boundary in a form they can enforce or audit.

4.2 Assumptions

- the user intentionally and legitimately launches the agent for a specific task;
- the agent has valid, correctly provisioned access to its assigned workspace and tools;
- the cluster control plane, scheduler, and any benchmark or monitoring harness are trusted;
- the attacker does not have root access and does not modify model weights;

- the attacker’s only foothold is the ability to influence content the agent will read, execute, or receive during normal operation, such as a file, a log, a tool’s output, shared state, or a message from another agent.

4.3 Attacker capability classes

Table 1 lists the attacker footholds considered here. Stating the foothold matters because an attack that already assumes unauthorized filesystem or administrator access demonstrates a conventional access-control failure, not the failure mode defined above.

Table 1: Attacker capability classes considered by the hijacked authorized agent threat model.

Class	Capability	Representative HPC source
C1	Write or influence a scientific artifact the agent will read	simulation log, metadata field, README, result file
C2	Influence scheduler-adjacent output	job stdout/stderr, wrapper diagnostics, accounting note
C3	Control a tool’s description or output	build helper, converter, module description, MCP tool server
C4	Write to shared node or filesystem state	scratch space, cache, shared collaboration path
C5	Control or hijack a peer agent	inter-agent message, staged intermediate artifact

5 Attack Surfaces in HPC

Prompt injection, poisoned tool output, and protocol abuse describe how an agent is manipulated. For HPC, we must also ask where the malicious content enters the workflow and what the redirected agent can affect. The following taxonomy is organized around those HPC surfaces. Its categories can lead to different harms, including data disclosure, allocation abuse, and corruption of scientific results, so they should not be treated as interchangeable instances of prompt injection.

5.1 Shared parallel-filesystem poisoning

Parallel filesystems and scratch spaces preserve state across jobs and, depending on site policy, expose shared or collaborative paths to several users. Malicious text placed in a result file, cache, README, or predictable output path may remain until a later agent reads it. The writer and reader need not run at the same time, which makes this a persistent workflow channel rather than a one-session interaction.

5.2 Scheduler-induced co-location and log injection

Agents that diagnose failures routinely inspect standard output, standard error, accounting records, and wrapper diagnostics. These inputs cannot simply be discarded as untrusted because they contain the evidence needed to repair a job. If an attacker can influence a related job or a shared output path, the resulting text enters a context in which scheduler commands are likely to be available. A successful redirection could therefore consume an allocation by submitting, resizing, or repeatedly resubmitting jobs.

5.3 Tool, module, and MCP poisoning

HPC software stacks include site modules, group-maintained build helpers, post-processing scripts, and, increasingly, MCP tool servers. Trust is often based on a mixture of site policy and local convention. Shi et al. show that a crafted tool description, not only executable output, can bias an agent’s tool selection [9]. In an HPC workflow, a module description or MCP manifest may therefore influence the agent before the selected program runs.

5.4 Cross-project data leakage

Researchers often belong to several projects, with access to separate allocations and datasets. An agent working on one project inherits whatever broader access the user exposes to it. A prompt-injection attack can exploit this mismatch by directing the agent to read from another project and disclose the result through an otherwise approved channel. Account-level permissions will not flag the read when the user is a legitimate member of both projects.

5.5 Coordinated multi-agent exfiltration

Multi-agent workflows pass messages and intermediate artifacts between planners and workers [2,4]. If one agent or artifact is compromised, it can steer a second agent that has different access. The resulting disclosure may be spread across several ordinary-looking actions: one agent writes an artifact, another reads protected data, and a third sends context to an approved model backend. Detecting the sequence requires more than reviewing each agent’s log in isolation.

6 Limits of Existing Controls

Table 2 compares established HPC and agent-monitoring controls with the information needed to recognize a hijacked agent. These controls still reduce risk; the issue is that they operate at a different boundary. Authentication, permissions, quotas, and isolation describe what an account may do. They do not record what a particular delegated task was meant to do. Accounting logs, ACLs, and network allowlists therefore cannot reconstruct task intent after an incident unless the agent or its runtime records that intent and links it to subsequent actions.

Table 2: Information captured by existing controls and information needed to recognize a hijacked agent.

Existing control	What it verifies	What it misses for hijacked agents
Authentication / SSO / zero-trust identity [14]	The identity and attributes of the user	Whether an authenticated action originated in the user’s request or in injected content
POSIX permissions, zone-based access architecture [12]	Resources the account may access	Whether those resources belong to the current task
Scheduler accounting and quotas	Resource use attributed to an account	Whether that use was requested or induced by poisoned content
Enforced user/process/network separation [13]	Isolation between users	Redirection within one user’s authorized session
Filesystem/network auditing, DLP	Data movement over observed channels	Disclosure through an approved channel, such as a normal LLM API request
Per-agent action logging	The actions selected by the agent	Whether an action follows task intent or adversarial context

7 Research Agenda

7.1 Security requirements for HPC-agent platforms

The comparison in Section 6 suggests several capabilities for an HPC-agent platform. Some belong in the agent runtime and others may require support from the scheduler, storage system, or site network. Their feasibility will vary across facilities, but together they define a useful design target:

- **Task-scoped authority.** The runtime should distinguish resources needed for the current task from resources available to the user’s account as a whole.
- **Context provenance.** User instructions, file content, tool output, and peer-agent messages should retain source and trust metadata when they enter the agent’s context.
- **Egress control.** Agent-to-model traffic needs policy enforcement because an approved, encrypted LLM API request can also carry protected data.
- **Shared-filesystem hygiene.** Sites need ways to limit or label stale and cross-user content in scratch, cache, and collaboration paths used by agents.
- **Tool and module trust.** Software-supply-chain checks should cover tool descriptions and MCP manifests as well as executable code.
- **Scheduler-aware containment.** Limits should prevent a redirected agent from turning a context compromise into excessive spending of the user’s allocation.
- **Cross-agent correlation.** Monitoring should connect related actions across agents so that a staged disclosure does not appear as several unrelated events.

7.2 Toward a benchmark: TaskBound

The claims above need empirical support. We are developing a companion benchmark called **TaskBound** to translate the threat model into executable tasks and attacks. The design follows AgentDojo’s practice of measuring task utility together with attack success [8], but replaces web and office applications with HPC-specific resources. Planned scenarios include Slurm actions, project-scoped filesystem policies, and checks for silent changes to scientific parameters, filters, or provenance. Each scenario pairs a user task with an explicit task policy and a deterministic security oracle. This should make it possible to compare agents and defenses on three practical questions: whether the agent completes the assigned work, whether adversarial context redirects it, and whether a defense improves security without making the workflow unusable.

7.3 Open questions

The benchmark will address only part of the larger problem. Several design questions also deserve study:

- How should users specify task scope when legitimate scientific work is exploratory, and how sensitive are security results to that specification?
- Do provenance labels and tool allowlists developed for web or enterprise agents transfer to scheduler and filesystem operations?
- Should task policy be enforced by the agent framework, scheduler, filesystem, or a separate monitor, especially at sites that cannot modify all four?
- How can a multi-agent workflow preserve useful delegation while preventing one agent from silently extending another agent’s authority?

8 Conclusion

LLM agents already interact with HPC software, filesystems, and schedulers [1, 2, 4, 5]. Their usefulness depends on reading the same logs, tools, and shared artifacts that can carry adversarial content. When such content redirects an agent, account-level security may see only valid credentials issuing permitted commands. We use *hijacked authorized agent* for this failure mode and argue that its defining boundary is the delegated task rather than the user account. The taxonomy and control analysis in this paper provide a starting point, but measurement is still missing. TaskBound is intended to supply that evidence by testing agents and defenses against realistic scheduler, storage, and scientific-integrity policies. The immediate question is not whether all HPC agents are unsafe. It is whether deployments can state what an agent was allowed to do and detect when its behavior departs from that task.

AI Disclosure

AI-assisted tools were used to identify literature, check citations, and support drafting. The authors reviewed and revised the manuscript and take responsibility for its claims and conclusions.

References

- [1] Heng Ma, Alexander Brace, Carlo Siebenschuh, Greg Pauloski, Ian Foster, and Arvind Ramanathan. Connecting large language model agent to high performance computing resource, 2025.
- [2] Daniel Rosendo, Stephen DeWitt, Renan Souza, Phillipe Austria, Tirthankar Ghosal, Marshall McDonnell, Ross Miller, Tyler Skluzacek, James Haley, Bruno Turcksin, Jesse McGaha, Benjamin Mintz, Feiyi Wang, Mallikarjun Shankar, Sarp Oral, and Rafael Ferreira da Silva. AI agents for enabling autonomous experiments at ORNL’s HPC and manufacturing user facilities. In *Proceedings of the SC’25 Workshops (SCW ’25)*, 2025.
- [3] William Dawson, Louis Beal, Yoann Curé, Giuseppe Fisicaro, Dorian Rolland, and Luigi Genovese. LARA: Validation-driven agentic supercomputer workflows for atomistic modeling, 2026.
- [4] Sean Mondesire, Emmanuel Nsiye, Bulent Soykan, and Glenn Martin. Automating HPC software compilation, deployment, and error resolution through an LLM-based multi-agent system. In *Practice and Experience in Advanced Research Computing (PEARC ’25)*, 2025. Best Paper, Systems and System Software track.
- [5] Aalto Scientific Computing. AI agents on HPC. <https://scicomp.aalto.fi/triton/usage/ai-agents/>, 2026. Aalto Triton user documentation; accessed 2026.
- [6] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec ’23)*, 2023. arXiv:2302.12173.
- [7] Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. Identifying the risks of LM agents with an LM-emulated sandbox. In *International Conference on Learning Representations (ICLR)*, 2024. Spotlight; arXiv:2309.15817.
- [8] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2406.13352.
- [9] Jiawen Shi, Zenghui Yuan, Guiyao Tie, Pan Zhou, Neil Zhenqiang Gong, and Lichao Sun. Prompt injection attack to tool selection in LLM agents. In *Network and Distributed System Security Symposium (NDSS)*, 2026. arXiv:2504.19793.
- [10] Mohamed Amine Ferrag, Merouane Debbah, Leandros Maglaras, et al. From prompt injections to protocol exploits: Threats in LLM-powered AI agents workflows, 2025.
- [11] Peiran Wang, Xinfeng Li, Chong Xiang, Jinghuai Zhang, Ying Li, Lixia Zhang, Xiaofeng Wang, and Yuan Tian. The landscape of prompt injection threats in LLM agents: From taxonomy to analysis, 2026.
- [12] National Institute of Standards and Technology. Special publication 800-223: High-performance computing (HPC) security: Architecture, threat analysis, and security posture. Technical report, NIST, 2024.

- [13] Andrew Prout, Albert Reuther, Michael Houle, Michael Jones, Peter Michaleas, LaToya Anderson, William Arcand, Bill Bergeron, David Bestor, Alex Bonn, Daniel Burrill, Chansup Byun, Vijay Gadepally, Matthew Hubbell, Hayden Jananthan, Piotr Luszczek, Lauren Milechin, Guillermo Morales, Julie Mullen, Antonio Rosa, Charles Yee, and Jeremy Kepner. HPC with enhanced user separation. In *S-HPC Workshop, SC'24*, 2024. arXiv:2409.10770.
- [14] Sadaf R. Alam, Christopher Woods, Matt Williams, Dave Moore, Isaac Prior, Ethan Williams, Anna Price, James Womack, Simon McIntosh-Smith, Fan Yang-Turner, Matt Pryor, and Ilja Livenson. Federated single sign-on and zero trust co-design for AI and HPC digital research infrastructures. In *Proceedings of the SC'24 Workshops (SCW '24)*, 2024. arXiv:2410.18411.